# Advanced Data Management
# Mock Midterm: Full Solutions

**Every answer worked and justified, with a map to the textbook chapters and a primer on the one topic the exam uses before the course reaches it**

Advanced Data Management

Mock exam dated 2026-10-12, 100 points, 2 hours

## How to use this document

The mock has three parts: five multiple-choice questions where the justification is worth as much as the letter, six discussion questions, and seven hands-on questions on a deliberately nasty database called `shopstream`. Each solution below gives the answer the way the grading rubric wants it: the number or the choice first, then the reasoning in the words of the course, then the trap. A box after each problem says where in the five textbook chapters the method lives. Part 4 at the end is a two-page primer on buffer management, because three questions (1.3, 2.6, 3.6) and part of 2.1 test Chapter 6, which the mock assumes and the lecture has not yet reached. All arithmetic was checked by script.

| Part | Content | Covered by chapters 1–5 | Points |
|---|---|---|---|
| 1 | Multiple choice | 4 of 5 (1.3 is Chapter 6) | 10 |
| 2 | Discussion | 5 of 6 (2.6 is Chapter 6; 2.1 half) | 45 |
| 3 | Hands-on | 6 of 7 (3.6 is Chapter 6; 3.2(b) needs a tool detail) | 45 |

## Part 1: Multiple choice (10 points)

**Problem: 1.1 External merge sort: $B = 800$, $M = 21$. Passes?** [2 pts]

(a) 2 (b) 3 (c) 4 (d) 39

**Solution**

**(b) 3 passes.** Phase 0 makes $\lceil 800/21 \rceil = 39$ sorted runs of 21 pages (one pass). Each merge pass combines $M - 1 = 20$ runs at a time: $39 \to 2 \to 1$, that is $\lceil \log_{20} 39 \rceil = 2$ merge passes. Total $1 + 2 = 3$ passes, about $2 \times 800 \times 3 = 4800$ page I/Os. Trap: (d) 39 is the number of runs, not

passes; (a) would need $\lceil 800/M \rceil \leq M - 1$, which fails ($39 > 20$).

**Problem: 1.2 Selectivity: 500,000 rows, 5,000 pages, B-tree on `amount`, query keeps 40%.** [2 pts]

(a) index scan (b) sequential scan (c) index-only scan, always (d) cannot say

**Solution**

**(b) sequential scan.** 40% of 500,000 is 200,000 rows. An index scan fetches each matching row with a random page read (cost 4 in the planner's units): $200{,}000 \times 4 = 800{,}000$ against one sequential pass of 5,000 pages. Even a bitmap heap scan, which reads each page once in order, would touch essentially all 5,000 pages (100 rows per page, 40% kept: every page has matches). The index pays only below roughly $1/(4 \cdot \text{rows per page}) = 0.25\%$, a few percent with bitmap scans. Traps: (c) index-only scans need a covering index and a fresh visibility map, and still read 200,000 index entries; (d) the fraction is given, so $X$ is irrelevant.

**Problem: 1.3 Buffer replacement: 3 frames, LRU, requests A B C A B D A B. Hits?** [2 pts]

(a) 2 (b) 3 (c) 4 (d) 5

**Solution**

**(c) 4 hits.**

| Request | A | B | C | A | B | D | A | B |
|---|---|---|---|---|---|---|---|---|
| Result | miss | miss | miss | hit | hit | miss, evict C | hit | hit |
| Frames, LRU first | A | AB | ABC | BCA | CAB | ABD | BDA | DAB |

When D arrives the pool is full; the least recently used page is C (A and B were both touched after it), so C is evicted. A and B then hit again. Four hits in eight requests. The trap is evicting A (first loaded) instead of C: that is FIFO, which would give 2 hits.

**Problem: 1.4 B+ tree, 2,000,000 rows, fanout 200: page reads for one point lookup, heap included.** [2 pts]

(a) 3 (b) 4 (c) 5 (d) 21

**Solution**

**(b) 4.** Leaves: $2{,}000{,}000/200 = 10{,}000$ leaf pages. Level above: $10{,}000/200 = 50$ internal pages. Above that: one root. Height 3 (root, internal, leaf), so 3 index pages plus 1 heap page for the row: 4 reads. Formula: $h = \lceil \log_{200} 2{,}000{,}000 \rceil = 3$ since $200^2 = 40{,}000 < 2 \times 10^6 <$

$200^3 = 8 \times 10^6$. Trap: (d) $21 \approx \log_2 2{,}000{,}000$ is binary search, not a tree with fanout 200. In practice the root and the internal level are always in the buffer pool, so the disk cost is closer to 2.

**Problem: 1.5 Block nested-loop:** $B_R = 300$, $B_S = 3{,}000$, $M = 32$. [2 pts]

(a) 9,300 (b) 30,300 (c) 900,300 (d) 3,300

**Solution**

**(b) 30,300.** Chunks of the outer: $\lceil 300/(32-2) \rceil = 10$. Cost $= 300 + 10 \times 3{,}000 = 30{,}300$ page reads. Traps: (a) 9,300 uses chunks of 100 pages (an $M$ of 102); (c) 900,300 is the *simple* nested loop with $|R| = 300$ rows instead of pages, $300 + 300 \times 3000$; (d) 3,300 is $B_R + B_S$, the cost when the whole outer fits in one chunk, which needs $M \geq 302$, or a hash join.

**Where it is in the textbooks**

1.1: Chapter 4, §4 (external merge sort, the passes formula and the $B = 1000$, $M = 11$ exercise). 1.2: Chapter 3, §5 (choosing indexes) and Chapter 5, §5.2 (access path from the cost model). 1.3: **Chapter 6, not yet lectured**; see the primer in Part 4 and Chapter 2 §6.2 for why the pool exists. 1.4: Chapter 3, §3 (B+ tree height, $h+1$ page reads for a lookup). 1.5: Chapter 4, §5.2 (block nested-loop) and its "common mistakes" box ($|R|$ versus $B_R$).

## Part 2: Discussion (45 points)

**Problem: 2.1 Why the DBMS runs its own buffer pool** [7 pts]

Three concrete reasons, what would break with the OS alone, one connected to durability.

**Solution**

1. **The DBMS knows the access pattern; the OS does not.** It knows that a sequential scan touches each page once, that a B+ tree root is touched by every lookup, that the next page of a scan is predictable (prefetch). With the OS alone, its generic least-recently-used policy would let one large scan flush every hot page out of memory (sequential flooding, 2.6), and index roots would be re-read from disk after every big query. The DBMS uses scan-resistant replacement (ring buffers, clock with usage counts) and pins pages it is working on.
2. **Durability needs control over *when* and *in what order* pages reach disk.** The write-ahead rule says the log record describing a change must be on disk before the changed data page is, and at commit

the log must be forced. The OS writes dirty file pages whenever it likes, in any order, and may write a data page before its log record. After a crash the disk could then hold a modified page with no record of how to undo it, or a committed transaction whose changes exist only in lost memory. With its own pool the DBMS decides which dirty page is written, when, and after which `fsync` of the log.

3. **Pinning and memory accounting.** An operator working on a page (splitting a B+ tree node, building a hash table) must be sure the page stays in memory until it is done; the OS could page it out in the middle, costing a disk round trip at the worst moment or, with memory-mapped files, a page fault inside a critical section. The DBMS also needs to budget memory per operator (`work_mem` for sorts and hashes versus the shared pool) and to know exactly which pages are in memory for cost estimation; the OS exposes none of that.

A fourth, if asked: avoiding double buffering (a page cached once by the OS and once by the DBMS wastes half the memory), which is why PostgreSQL keeps `shared_buffers` modest and still benefits from the OS cache behind it.

**Problem: 2.2 Why we count pages, not instructions** [7 pts]

Why page reads are the metric; one situation where CPU dominates; how to notice it.

**Solution**

**Why pages.** A page read from disk takes milliseconds on a spinning disk and tens of microseconds on an SSD; an instruction takes a nanosecond. One I/O is worth $10^4$ to $10^6$ instructions, so for any operator that touches disk the I/O term dwarfs the CPU term, and a cost model that counts only pages is within a few percent of the truth while being simple, hardware-independent and computable from catalog statistics alone ($B_R$, $|R|$, $M$). The planner's own weights encode the same judgement: `seq_page_cost` 1.0 against `cpu_tuple_cost` 0.01.

**Where it misleads.** When the data is already in memory and the per-tuple work is heavy. Examples: a `GROUP BY` with a hash aggregate over a table that fits in the buffer pool, a filter with a regular expression or `LIKE '%x%'` on a wide text column, extracting fields from `jsonb` payloads, sorting millions of in-memory rows, or an expensive user-defined function. There, page count says "cheap" and the query runs for minutes on one core.

**How to notice it in `EXPLAIN (ANALYZE, BUFFERS)`.** The `Buffers` line shows `shared hit` high and `read` near zero, yet `actual time` is large; the time is concentrated in a node that does no I/O (a `HashAggregate`, a `Sort` with `Sort Method: quicksort Memory:`, a `Seq Scan` whose `Filter` is a regex) rather than in the scans; and the rows processed per millisecond are far below what a page-bound scan achieves. Running the query twice makes it obvious: a page-bound query gets much faster when warm, a

CPU-bound one does not.

**Problem: 2.3 "Index everything"** [8 pts]

Consequences for reads, writes, space and the optimizer; why some indexes are never used (selectivity); how to decide which to keep.

**Solution**

**Reads.** A few queries get faster: point lookups and selective ranges on the indexed columns, and joins where that column is the inner key. Most reads do not change, because most columns of a large table are not what queries filter on.

**Writes.** Every `INSERT` now writes the heap page plus one leaf page per index, so a table with $n$ indexes does about $n+1$ page writes per row instead of one, at random positions (keys hash or sort to scattered leaves), which defeats sequential writing. Every `UPDATE` of an indexed column deletes and re-inserts an index entry, and PostgreSQL loses its cheap "heap-only tuple" update path whenever any indexed column changes. `DELETE` leaves dead entries in every index for vacuum to clean. On a write-heavy table this is the dominant cost: throughput falls roughly in proportion to the number of indexes (Lab 3 measured 2–4× for three indexes).

**Space.** A B+ tree stores the key plus a row pointer per row, about 16 bytes for an integer key and far more for text; an index on a wide column can be as large as the table. Indexing every column of a table with twenty columns can multiply the footprint several times, and all of it competes for the buffer pool with the data itself.

**Optimizer.** More indexes mean more candidate access paths to cost for every query, so planning is slower, and more opportunities for a wrong estimate to make a bad index look attractive. Vacuum and `ANALYZE` also take longer.

**Never used, by selectivity.** The planner uses an index only when the predicate keeps a small fraction of the rows, below about $1/(4 \cdot \text{rows per page})$ for a plain index scan and a few percent for a bitmap scan, because each matched row costs a random page read against a sequential pass over the table. A column with few distinct values (`status`, a boolean, `event_type` where `'view'` is 90%) has selectivity $\geq 0.1$ for most predicates, so an index on it is never the cheaper path and the planner correctly ignores it. It then pays full write and space cost for nothing.

**Deciding.** Start from the workload: list the queries that matter and what they filter, join and sort on. Keep indexes that a query plan actually uses (`EXPLAIN` before and after) and that return few rows; keep primary and foreign keys and the join columns; prefer one composite index that serves several queries (leftmost-prefix rule) over many single-column ones; use partial indexes for narrow predicates like `event_type = 'purchase'`. Then measure: `pg_stat_user_indexes.idx_scan = 0` after a representative period means nobody uses the index; drop it.

**Problem: 2.4 Three joins, kept on purpose** [8 pts]

Best case and disaster for each; why keep all three.

**Solution**

| Join | Clearly best | Disaster |
|---|---|---|
| Nested loop | A tiny outer and an index on the inner's join key: $B_R + \|R\| \cdot (h + \text{matches})$; nine customers against a million orders in under a millisecond. Also the only one that handles arbitrary conditions (`<`, `LIKE`). | Two large tables with no index: $B_R + \lceil B_R/(M-2) \rceil B_S$, the inner re-read once per chunk of the outer; or a nested loop chosen because the outer was *estimated* tiny and was not (the mock's 13-minute query). |
| Sort-merge | Inputs already sorted on the key (an index scan delivers them so), or sorted output wanted anyway (`ORDER BY`, `GROUP BY` on the key), or inequality joins: then the merge is one linear pass. | Both inputs large, unsorted, with little memory: two multi-pass external sorts, about $2B \times$ passes each, before any output, when a hash join would read each input once. |
| Hash | A large equi-join with no useful order and a build side that fits in memory: $B_R + B_S$, the minimum possible. | A non-equality condition (hashing finds equal keys only); a build side far larger than memory (Grace partitioning, $3(B_R + B_S)$ and many batches); heavy skew on the build key (one bucket holds most rows). |

**Why keep all three.** No algorithm dominates: the winner depends on the join condition (equality or not), the sizes (is one side tiny?), the indexes (is there one on the inner key?), the order already present or required, and the memory available for a build side or a sort. The optimizer costs all three for every join with the estimated sizes and picks per query; the lecture's class example ran the same 500,000-row join three ways (hash 232 ms, merge 432 ms, nested loop 641 ms) and then watched the choice flip to nested loop when a filter shrank the outer to nine rows. Dropping two of them would make some queries orders of magnitude slower or impossible to execute.

**Problem: 2.5 The hardest part of optimization** [8 pts]

Defend the claim; how a wrong leaf estimate propagates; two reasons estimates go wrong, with fixes.

**Solution**

**The claim.** Rewriting is mechanical and always safe; costing is arithmetic on the Chapter 4 formulas; join enumeration is a solved search

problem. Every one of those takes *row counts* as input, and the row counts for anything but a base table must be predicted from summaries of the data: selectivity from `n_distinct` and histograms, join sizes from distinct counts, conjunctions by multiplying. Predicting how many rows a real, skewed, correlated, changing dataset produces is the only genuinely uncertain step, and the usual root cause of a bad plan is a bad estimate, not a bad search.

**Propagation.** Plans are costed bottom-up. If a leaf filter is estimated at 40 rows when the truth is 200,000 (5000×), then: the join above it is estimated at 5000× too few rows; the optimizer chooses a *nested loop* because a 40-row outer makes it cheapest, and the real execution loops 200,000 times over the inner; the *join order* is chosen to bring that "tiny" result in first, so the largest intermediate lands where the optimizer expected the smallest; memory for the build side or the sort is sized for 40 rows and spills; the access path for the inner may be a scan when an index would have paid. Errors do not cancel; through each join they multiply, so a factor at the leaf becomes the same factor on every node above and a different algorithm at each of them.

**Two reasons and fixes.** (1) *Stale statistics*: the data changed since the last `ANALYZE` (bulk loads, a table that grew 10×); every estimate is scaled to the old row count and the old histogram. Fix: `ANALYZE` the table, and tune autovacuum so it analyses large tables sooner. (2) *Correlated columns*: the independence assumption multiplies selectivities, but `city = 'Casablanca'` already implies `country = 'MA'`, so the product underestimates by the strength of the dependency. Fix: `CREATE STATISTICS ... (dependencies) ON country, city` then `ANALYZE`. A third, if asked: skew, where a dominant value is not in the MCV list; fix by raising the column's statistics target.

## Problem: 2.6 Sequential flooding and the steal policy [7 pts]

(a) What flooding is, why LRU is vulnerable, how real systems defend.
(b) Define steal and no-force; why they force the database to keep a log.

## Solution

**(a) Sequential flooding.** A large sequential scan reads every page of a table once and never again. Under LRU each page it reads becomes the most recently used, so the scan's pages push out every page that was useful (index roots, hot rows of small tables, other queries' working sets). When the scan is done the pool is full of pages nobody will touch, and every other query misses until its working set is reloaded. LRU is vulnerable because it treats "touched once, just now" exactly like "touched a hundred times"; recency is the only signal, and a scan maximises recency for useless pages. Worse, if the scanned table is larger than the pool, LRU gets a *zero* hit ratio even when the same scan is repeated: each page is evicted before the next pass returns to it.

**Defences.** PostgreSQL gives a sequential scan of a table larger than a quarter of `shared_buffers` its own small **ring buffer** (256 KB, 32 pages) that the scan recycles, so the main pool is untouched; bulk writes and `VACUUM` get rings too. Its replacement policy is a **clock sweep** with a usage counter per page, so a page touched once has low priority compared with one touched often. Other systems use LRU-K or 2Q (only promote a page after its second reference), MRU for scans, or priority hints from the operators.

**(b) Steal and no-force.** *Steal*: the buffer manager may write a dirty page containing an *uncommitted* transaction's changes to disk, typically because it needs the frame for something else. *No-force*: at commit, the transaction's dirty pages are *not* required to be written to disk; they may stay in memory and be flushed later. Both are pure performance choices: steal lets eviction pick any frame, no-force makes commit cheap (no burst of random page writes per transaction).

**Why they force a log.** Steal means that after a crash the disk may contain changes of a transaction that never committed; to restore atomicity the system must be able to *undo* them, so before a dirty page is written the old values must be safely on disk somewhere: the log, written first (write-ahead). No-force means that after a crash a committed transaction's changes may exist nowhere but in lost memory; to guarantee durability the system must be able to *redo* them, so at commit the new values must be on disk somewhere cheaper than the pages themselves: again the log, sequentially appended and forced once per commit. The log is the price of steal plus no-force, and the WAL rule (log record before data page, log forced at commit) is exactly the schedule that makes undo and redo possible.

**Where it is in the textbooks**

2.1: Chapter 2, §6.2 (why the DBMS manages its own pages, with the write-ordering argument) and Chapter 1 §2.3; the flooding half is Chapter 6 (Part 4). 2.2: Chapter 2, §2 (the I/O cost model) and Chapter 5, §5.1 (cost weights) and §7.5 (reading `Buffers`). 2.3: Chapter 3, §5 (helps versus hurts, the cost of having an index, design discipline) and Chapter 5 §5.2. 2.4: Chapter 4, §5 (the three joins and "which join wins") and its classic problem P7. 2.5: Chapter 5, §4.3 (estimates propagate), §7.2–7.4 (the failures) and classic problem P4. 2.6: **Chapter 6, not yet lectured**: Part 4 below; Chapter 2 §6.2 already states the write-ahead ordering.

## Part 3: Hands-on on `shopstream` (45 points)

Facts to keep in view: `users` 2,000,000 rows, 30,000 pages; `events` 50,000,000 rows, 1,000,000 pages, 50 rows per page; `shared_buffers` 500,000 pages; `work_mem` 8,000 pages; 500 MB/s sequential; statistics last collected at 5M events; `city` determines `country`.

**Problem: 3.1 Diagnose the 13-minute plan** [6 pts]

Nested Loop `rows=50` actual 1,240,000; Seq Scan on users `rows=40` actual 200,000; inner Seq Scan on events per outer row. (a) Most wrong node and its number. (b) Two root causes. (c) Commands to fix it, and the join the optimizer should choose.

**Solution**

**(a)** The leaf `Seq Scan on users` with `Filter: city = 'Casablanca'`: estimated 40 rows, actual 200,000, a factor of 5,000. (The `Nested Loop` above it is off by 1,240,000/50 $\approx$ 24,800×, but that is the leaf's error multiplied by the join; the leaf is where it starts, which is why the rubric wants the lowest wrong node.) The consequence is visible in the plan shape: the optimizer, believing 40 outer rows, chose a nested loop whose inner side is a *sequential scan of events, 1,000,000 pages, once per outer row*: 200,000 $\times$ $10^6$ page reads if it ran to the end.

**(b)** Two root causes. (1) *Stale statistics*: `ANALYZE` last ran when `events` had 5M rows; it is now 50M, and the `city` distribution of `users` is either unknown or old, so the planner falls back to a near-default selectivity for `city = 'Casablanca'` instead of the true 10%. (2) *Correlation and skew the planner cannot see*: `city` determines `country` (so any estimate that multiplies them is 5× too low), and `event_type` is heavily skewed, so `event_type = 'purchase'` needs the MCV list to be estimated at 0.5% rather than one third of "some distinct values".

**(c)** Commands, in order:

```
ANALYZE users;                          -- fresh counts, MCVs for
    city
ANALYZE events;                         -- 50M rows, event_type MCVs
CREATE STATISTICS s_geo (dependencies) ON country, city FROM
    users;
ANALYZE users;                          -- fill the extended
    statistics
ALTER TABLE events ALTER COLUMN event_type SET STATISTICS 1000;
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;    -- rows vs actual rows
```

The optimizer should now choose a **hash join**: build the hash table on the filtered `users` (200,000 rows, about 3,000 pages, which fits in `work_mem` of 8,000 pages, so `Batches: 1`), probe it with a single sequential scan of `events` filtered on `event_type = 'purchase'`, then a `HashAggregate` on `country`. Cost about 30,000 + 1,000,000 pages, read once each; at 500 MB/s that is roughly 16 seconds for `events` instead of 13 minutes. A partial index `ON events(user_id) WHERE event_type = 'purchase'` would let an index nested loop over the 1.24M purchase rows compete, but the hash join is the safe answer with no index present.

**Problem: 3.2 Storage and columnar** [6 pts]

(a) Confirm 1,000,000 pages and the cold-scan time at 500 MB/s. (b) Export `ts` to Parquet and count rows per day in DuckDB; why cheaper. (c) Verify rows per page with `pg_relation_size` and `pageinspect`.

**Solution**

**(a)** 50,000,000 rows/50 rows per page = 1,000,000 pages; at 8 KB a page that is 8,000,000 KB ≈ 8 GB (7.6 GiB). Sequential throughput 500 MB/s gives 8,000/500 = 16 seconds (16.4 s with 8,192 MB). That is the floor for any plan that reads the whole table cold; the 13-minute query above spent its time re-reading it.

**(b)** PostgreSQL's `COPY` writes text or binary, not Parquet, so the export goes through a tool that does. With the `pg_parquet` extension installed:

```
COPY (SELECT ts FROM events) TO '/data/events_ts.parquet' (FORMAT
    parquet);
```

Without it, export CSV and let DuckDB convert, or let DuckDB read PostgreSQL directly:

```
-- PostgreSQL
COPY (SELECT ts FROM events) TO '/data/events_ts.csv' (FORMAT csv
    , HEADER);
-- DuckDB
COPY (SELECT * FROM read_csv_auto('/data/events_ts.csv'))
  TO '/data/events_ts.parquet' (FORMAT parquet);
SELECT date_trunc('day', ts) AS day, count(*) AS n
FROM read_parquet('/data/events_ts.parquet') GROUP BY day ORDER
    BY day;
```

Why it is far cheaper: the PostgreSQL scan reads every page of the row store, 8 GB, including the wide `jsonb` payload headers and all other columns, to look at one 8-byte field per row. The Parquet file holds only `ts`: 50M × 8 bytes = 400 MB raw, and timestamps that arrive in order compress with delta encoding to a few tens of megabytes. DuckDB then reads that one column with vectorised execution and can skip row groups by their min/max statistics. Roughly 20× less raw data, and 50–100× fewer bytes after compression: the row-versus-column argument of Chapter 2, measured.

**(c)**

```
SELECT pg_relation_size('events') / 8192 AS pages;
      -- expect ~1,000,000
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT count(*) FROM heap_page_items(get_raw_page('events', 0));
      -- rows on page 0, expect ~50
```

Rows per page = 50,000,000/pages; check it against the second query on a few pages (`get_raw_page('events', 12345)`). The large `payload` does not distort this because it lives out of line in the TOAST table; only an 18-byte pointer stays in the row.

**Problem: 3.3 Index design** [8 pts]

For each pattern: `CREATE INDEX`, type, one-line reason; the pattern no index helps, with a number.

**Solution**

(a) `CREATE UNIQUE INDEX ON users(email);` B-tree, unique. Selectivity 1/2,000,000: four page reads instead of a 30,000-page scan, and the constraint is enforced for free.

(b) `CREATE INDEX ON events(ts);` B-tree: a range walks the leaves in order. Because events are appended in time order, `CREATE INDEX ON events USING brin (ts);` is the cheaper alternative: a block-range index of a few hundred KB that lets the scan skip every page range outside `[$1, $2]`.

(c) `CREATE INDEX ON events(user_id) WHERE event_type = 'purchase';` partial B-tree. `'purchase'` is 0.5% of rows (250,000), so a bitmap scan beats reading 1,000,000 pages, and a partial index on just those rows is 200× smaller than a full one on `event_type`; keying it on `user_id` also serves the join in 3.1.

(d) **No index helps.** `'view'` is 90%: $0.9 \times 50{,}000{,}000 = 45{,}000{,}000$ rows on essentially every one of the 1,000,000 pages. An index scan would do 45M random fetches against one sequential pass of 1M pages; even a bitmap heap scan reads all 1M pages. Selectivity 0.9 is three orders of magnitude above the threshold ($1/(4 \times 50) = 0.5\%$), so the planner will sequential-scan, and it is right.

(e) `CREATE INDEX ON users(lower(email));` expression index. The B-tree on `email` is sorted by `email`, not `lower(email)`, so the predicate cannot use it; the expression index stores the lowered value.

(f) `CREATE INDEX ON events(user_id);` B-tree on the foreign key. It makes index nested-loop joins possible when the `users` side is small (one lookup per user: $h +$ matches pages), and it is what every "events of this user" query needs.

**Problem: 3.4 External sort and join cost** [8 pts]

(a) Sort events (1,000,000 pages) with $M = 8{,}000$: runs, passes, total I/O.
(b) Join events to users on `user_id` with no index: which join, its I/O, and whether the build side fits.

**Solution**

**(a)** Initial runs: $\lceil 1{,}000{,}000/8{,}000 \rceil = 125$ runs of 8,000 pages. Fan-in $M - 1 = 7{,}999 \geq 125$, so one merge pass finishes: passes $= 1 + \lceil \log_{7999} 125 \rceil = 2$. Total I/O $\approx 2B \times$ passes $= 2 \times 1{,}000{,}000 \times 2 = 4{,}000{,}000$ page I/Os (each pass reads and writes the data once: 2M reads, 2M writes, about 32 GB of traffic, over a minute at 500 MB/s). With $M \geq 1{,}000$ the two-pass regime is guaranteed, since $\sqrt{B} = 1{,}000$.

**(b) Hash join**, with `users` as the build side (the smaller input, 30,000 pages). Does it fit? `work_mem` is 8,000 pages and the build needs about 30,000 (plus hash-table overhead): **no**. So it is a Grace (multi-batch) hash join: partition both inputs by `user_id` into about $\lceil 30{,}000/8{,}000 \rceil = 4$ batches (PostgreSQL rounds to a power of two), then join batch by batch. I/O $\approx 3(B_R + B_S) = 3 \times (30{,}000 + 1{,}000{,}000) = 3{,}090{,}000$ pages. Compare: sort-merge needs the 4,000,000 I/Os of (a) plus sorting `users` (120,000) plus the merge (1,030,000): about 5,150,000; block nested loop with `users` outside: $30{,}000 + \lceil 30{,}000/7{,}998 \rceil \times 1{,}000{,}000 = 4{,}030{,}000$. Hash wins at 3.09M. The plan will show `Hash ... Batches: 4` under the join. If the query also filtered `users` (as in 3.1), the build side would shrink to about 3,000 pages, fit in one batch, and the cost would drop to $B_R + B_S \approx 1{,}030{,}000$.

## Problem: 3.5 Estimation failures [8 pts]

(a) `country = 'MA' AND city = 'Casablanca'` with selectivities 0.2 and 0.1: independence estimate, true count, the gap, the fix. (b) `'view'` versus `'purchase'`: selectivity, rows, index or scan.

## Solution

**(a)** Independence: $0.2 \times 0.1 \times 2{,}000{,}000 = 40{,}000$ rows. Truth: every Casablanca user is in MA, so the city predicate implies the country predicate and the conjunction keeps exactly the Casablanca users, $0.1 \times 2{,}000{,}000 = 200{,}000$ rows (the `actual rows=200000` of the plan in 3.1). The planner is 5× too low, by precisely the factor sel(country) = 0.2 that it multiplied in for nothing. Fix:

```
CREATE STATISTICS s_geo (dependencies) ON country, city FROM
    users;
ANALYZE users;
```

After which the planner knows the functional dependency `city` ⇒ `country` and estimates about 200,000.

**(b)** `event_type = 'view'`: selectivity 0.9, 45,000,000 rows. `'purchase'`: selectivity 0.005, 250,000 rows. Both come from the MCV list, which stores the three values with their frequencies; without fresh statistics the planner would guess 1/3 for each. An index serves `'purchase'`: 250,000 rows, at most 250,000 pages touched by a bitmap heap scan (and far fewer with the partial index of 3.3(c)), against 1,000,000 for a scan. The planner will scan for `'view'`: 45M matching rows touch every page, so no index can read fewer than the 1,000,000 pages a sequential scan reads, and an index would only add random I/O. Same column, same index, opposite decision: selectivity, not the existence of an index, decides the access path.

**Problem: 3.6 Buffer behaviour** [5 pts]

Pool 500,000 pages, cold scan of 1,000,000 pages. (a) Expected hit ratio and why. (b) What plain LRU does to other queries' pages, and what PostgreSQL does instead. (c) The `pg_buffercache` query.

**Solution**

**(a)** About **0%**. The scan is cold, so every page is read from disk exactly once: no page can hit on its first touch. Even a repeated scan would not help under LRU: the table is twice the pool, so by the time the scan returns to page 1 it has been evicted by pages 500,001 onward (the classic LRU pathology for a scan larger than memory). Hits can only come from pages of `events` that happened to be cached by earlier queries, and a 1M-page scan dwarfs those.

**(b)** Under plain LRU each of the 1,000,000 pages becomes most-recently-used as it is read, so the scan evicts the entire previous contents of the pool: every index root, every hot `users` page, every other session's working set. Those queries then miss for a while after the scan: sequential flooding. PostgreSQL prevents it by giving a sequential scan of a table larger than a quarter of `shared_buffers` (here 1M pages $\gg$ 125,000) a private **ring buffer** of 256 KB (32 pages) that the scan reuses, so the scan occupies 32 frames of the pool instead of 500,000; its clock-sweep replacement with usage counts also keeps once-touched pages from displacing frequently-touched ones.

**(c)**

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache;
SELECT c.relname, count(*) AS buffers, pg_size_pretty(count(*) *
    8192) AS size
FROM pg_buffercache b
JOIN pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE b.reldatabase IN (0, (SELECT oid FROM pg_database WHERE
    datname = current_database()))
GROUP BY c.relname ORDER BY buffers DESC LIMIT 10;
```

Run it before and after the scan: with the ring buffer, `events` appears with a few dozen buffers, not hundreds of thousands.

**Problem: 3.7 Write the SQL** [4 pts]

(a) Purchase events per country in the last 30 days. (b) One `generate_series` insert of 1,000,000 events with the 90 / 9.5 / 0.5 skew.

**Solution**

**(a)**

```
SELECT u.country, count(*) AS purchases
FROM events e JOIN users u ON u.user_id = e.user_id
WHERE e.event_type = 'purchase'
```

```
  AND e.ts >= date '2026-10-12' - interval '30 days'
GROUP BY u.country ORDER BY purchases DESC;
```

Both filters are on `events` and are pushed below the join; with the partial index of 3.3(c) or the `ts` index of 3.3(b) the scan side becomes an index or bitmap scan over the recent purchases.

**(b)** Draw one random number per row and cut it at 0.90 and 0.995:

```
INSERT INTO events (user_id, event_type, ts, product_id, payload,
    device)
SELECT 1 + floor(random() * 2000000)::int,
       CASE WHEN r < 0.90 THEN 'view' WHEN r < 0.995 THEN 'cart'
    ELSE 'purchase' END,
       timestamp '2026-10-12' - random() * interval '365 days',
       1 + floor(random() * 50000)::int,
       '{}'::jsonb,
       (ARRAY['web','ios','android'])[1 + floor(random() * 3)::
    int]
FROM (SELECT random() AS r FROM generate_series(1, 1000000)) g;
```

The subquery matters: computing `random()` once as `r` and comparing it twice keeps the three branches consistent; calling `random()` inside each `WHEN` would draw different numbers and skew the proportions. `event_id` is omitted so its default fills in. After the insert, remember `ANALYZE events;`, which is the whole point of 3.1.

**Where it is in the textbooks**

3.1: Chapter 5, §7.2, §7.3 and §7.5 (the two failures, the diagnosis technique); the fixed plan is Chapter 4 §5.4. 3.2: Chapter 2, §2 (pages and throughput), §5 (row versus column, with DuckDB and Parquet as the column-store examples) and §6.1 (`pg_relation_size`, `pageinspect`, TOAST). The `COPY ... (FORMAT parquet)` tool detail is not in the chapters. 3.3: Chapter 3, §5 and §6 (index types: B-tree, partial, expression, BRIN, covering; the four scan types). 3.4: Chapter 4, §4 (sort passes) and §5.4 (hash join, Grace partitioning, `Batches`); the comparison table in §5.5. 3.5: Chapter 5, §4.1 (selectivity rules), §7.3 (correlation) and §7.4 (skew and the MCV list). 3.6: **Chapter 6, not yet lectured**: Part 4 below. 3.7: Chapter 1, §5 (SQL refresher) and the lab's `generate_series` inserts (Lab 5, Part 4).

## Coverage summary

**The five chapters cover 84 of the 100 points.** Everything on sorting, joins, indexes, storage, statistics, estimation and plan diagnosis is in Chapters 2 to 5 at the depth the mock needs, with the same formulas and the same worked numbers. The missing 16 points are of two kinds:

- **Buffer management (14 points: 1.3, 2.6, 3.6).** Replacement policies

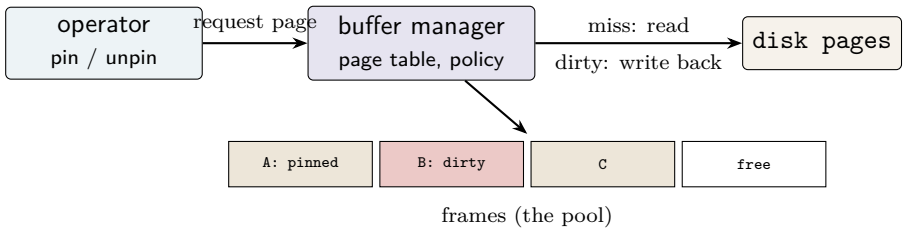
and the LRU trace, sequential flooding and ring buffers, steal and no-force and why they need a log, hit ratios, `pg_buffercache`. The lecture deck ends with "Next: Chapter 6, Buffer management", so the course will cover it; the mock assumes it already. Part 4 below gives exactly what these questions need.

- **One tool detail (about 2 points in 3.2(b)).** Exporting a column to Parquet and querying it in DuckDB. Chapter 2 makes the row-versus-column argument and names both tools, but not the exact export and `read_parquet` syntax; the solution above has it.

## Part 4: Buffer management primer (what the mock assumes from Chapter 6)

**Concept: The buffer pool**

The buffer pool is an array of **frames** in memory, each the size of one disk page (8 KB), managed by the DBMS itself for the reasons of 2.1. A **page table** maps page ids to frames. Each frame carries a **pin count** (how many operators are using the page right now; a pinned page cannot be evicted) and a **dirty bit** (the page was modified and must be written back before its frame is reused). Every page access goes through the pool: on a **hit** the page is already in a frame; on a **miss** a victim frame is chosen by the replacement policy, written back if dirty, and the page is read from disk into it. The **hit ratio** is hits divided by requests; a workload whose working set fits in the pool approaches 100%, a cold scan larger than the pool approaches 0%.




**Mechanism: Replacement policies**

**LRU**: evict the page whose last use is oldest. Good for temporal locality, defeated by scans (sequential flooding, 2.6). Exact LRU needs a list update on every access. **Clock** (second chance): frames in a circle with a reference bit; the hand advances, clearing set bits and evicting the first frame whose bit is already clear; an approximation of LRU with no list. PostgreSQL's **clock sweep** keeps a usage *counter* (up to 5) instead of a bit: each access increments it, the hand decrements, and a frame is evicted at zero, so a page touched many times survives several sweeps. **LRU-K / 2Q**: rank by the $K$-th most recent access, or admit a page to the main list

only on its second reference, so once-touched scan pages never displace hot pages. **MRU** for a scan that will repeat: evict the page just read, since it is the one needed furthest in the future. The theoretical optimum (evict the page needed furthest in the future) is unachievable without knowing the future; the DBMS gets close because its operators *do* know their own future, which is why scans announce themselves and get ring buffers.

**Worked example: Tracing a policy by hand**

Write the frames after every request, oldest on the left, and for each miss say what was evicted and why (1.3 is the template). For clock, also track the hand and the bits. Count hits, not misses; a question that says "starting empty" means the first touch of every page is a compulsory miss.

**Concept: Steal, force, and the log**

Two independent policy bits describe how the pool treats transactions. **Steal / no-steal**: may a dirty page of an uncommitted transaction be written to disk? **Force / no-force**: must a transaction's dirty pages be written at commit? *No-steal + force* needs no log at all (nothing uncommitted ever reaches disk; everything committed is on disk at commit) but is unusable: the pool could fill with pinned uncommitted pages, and every commit would burst random writes. Every real system runs *steal + no-force* for performance, and pays with a **write-ahead log**: undo information on disk before a dirty page is stolen, redo information forced at commit. Recovery after a crash then redoes committed work that was still in memory and undoes uncommitted work that was stolen to disk (ARIES). PostgreSQL's WAL is this log; `synchronous_commit` is the force-the-log-at-commit switch.

**Technique: PostgreSQL specifics the mock asks for**

- `shared_buffers`: the pool size (4 GB here, 500,000 pages). The OS page cache sits behind it; a miss in the pool may still be an OS-cache hit, which `EXPLAIN (ANALYZE, BUFFERS)` reports as `read` either way.
- Ring buffers: a sequential scan of a table larger than `shared_buffers`/4 uses a 256 KB ring; `COPY`, `CREATE TABLE AS` and `VACUUM` use rings of their own. Hence a 1M-page scan leaves the pool almost untouched.
- `pg_buffercache`: one row per frame (`relfilenode`, `relblocknumber`, `isdirty`, `usagecount`); the query in 3.6(c) aggregates it per table. Hit ratios per table come from `pg_statio_user_tables` (`heap_blks_hit` versus `heap_blks_read`).
- `pg_prewarm` loads a table into the pool on purpose; the background writer and checkpoints write dirty pages back so that eviction rarely has to.

**Summary: Buffer management in five lines**

1. The pool is an array of page-sized frames with a page table, pin counts and dirty bits; hits avoid disk, misses evict a victim.
2. LRU is defeated by scans (sequential flooding); clock sweep with usage counts, LRU-K/2Q and ring buffers for scans fix it.
3. A cold scan larger than the pool has a hit ratio near 0, under LRU even when repeated.
4. Steal + no-force is chosen for speed and forces a write-ahead log: undo before a steal, redo at commit.
5. PostgreSQL: `shared_buffers`, 256 KB rings for big scans, `pg_buffercache` to look inside.